

Ant Colony Optimization for the Job Shop Scheduling Problem

Updated Technical Report and Implementation Notes

Adam DeJans Jr.

June 2026

Repository: github.com/addejans/ACO-JSSP
Live demo: addejans.github.io/ACO-JSSP
Implementation: `src/aco_jssp.py`
Visualizer: `docs/index.html`

Abstract

This report updates the original ACO-JSSP project documentation to reflect the modern Python 3 solver, generated instances, browser-based Gantt visualizer, test suite, and GitHub Pages deployment. The project demonstrates how Ant Colony Optimization can be used as a constructive metaheuristic for the Job Shop Scheduling Problem. The current implementation separates problem parameters from algorithm parameters, supports arbitrary generated job-shop instances within bounded UI ranges, applies a remaining-work-aware transition heuristic, mixes exploration with exploitation, reinforces both iteration-best and global-best solutions, and applies pair-swap local search to improve constructed schedules. The objective is practical clarity: valid schedules, understandable controls, visible Gantt charts, and a codebase organized for future algorithmic extensions.

Contents

1	Introduction	2
2	Problem Definition	2
2.1	Job precedence	2
2.2	Machine capacity	2
2.3	Objective	2
3	Why Ant Colony Optimization?	3
4	Current Repository Organization	3
5	Solver Design	3
5.1	Generated problem instances	3
5.2	Job-order decoding	3
5.3	Transition rule	4
6	Additional Techniques Implemented	4
6.1	Separation of problem and algorithm parameters	4
6.2	Exploration and exploitation	4
6.3	Pheromone evaporation and reinforcement	5
6.4	Pair-swap local search	5
6.5	Seeded reproducibility	5
7	Browser Visualizer	5
8	Testing and Validation	5
9	Limitations	6
10	Future Work	6
11	Conclusion	6

1 Introduction

The Job Shop Scheduling Problem (JSSP) is a canonical problem in operations research and combinatorial optimization. A set of jobs must be processed on a set of machines. Each job has a fixed route through the machines, and each operation has a processing time. A feasible schedule must respect two core restrictions: job precedence and machine capacity. Job precedence means that a job's second operation cannot begin until its first operation is complete. Machine capacity means that a machine can process at most one operation at a time.

This repository began as a 2017 research prototype applying Ant Colony Optimization (ACO) to a small job-shop instance. The original version was useful as a learning artifact but difficult to maintain: it used Python 2 syntax, relied on an old Plotly API, used heavy global state, and did not include tests or a modern UI. The updated repository keeps the historical scripts for reference, but moves the active implementation into a cleaner Python 3 module with tests and a browser visualizer.

The live visualizer is available at:

addejans.github.io/ACO-JSSP

The visualizer lets a user choose the number of jobs, number of machines, operation duration range, ant count, iteration count, random seeds, and local-search budget. The output is a Gantt chart and a JSON representation of the best schedule found.

2 Problem Definition

Let $J = \{1, \dots, n\}$ be the set of jobs and $M = \{1, \dots, m\}$ be the set of machines. Each job j contains an ordered sequence of operations:

$$O_j = (O_{j,1}, O_{j,2}, \dots, O_{j,m}).$$

Each operation $O_{j,k}$ has a processing time $p_{j,k} > 0$ and a required machine $\mu_{j,k} \in M$. In the generated instances used by the visualizer, each job visits every machine exactly once, but in a randomly generated route order.

The goal is to assign a start time $s_{j,k}$ to every operation. A schedule is feasible if it satisfies:

2.1 Job precedence

$$s_{j,k+1} \geq s_{j,k} + p_{j,k} \quad \forall j, k = 1, \dots, m-1.$$

2.2 Machine capacity

For any two operations $O_{j,k}$ and $O_{j',k'}$ requiring the same machine, they cannot overlap:

$$[s_{j,k}, s_{j,k} + p_{j,k}) \cap [s_{j',k'}, s_{j',k'} + p_{j',k'}) = \emptyset.$$

2.3 Objective

The repository minimizes makespan:

$$C_{\max} = \max_{j,k} (s_{j,k} + p_{j,k}).$$

A smaller makespan means the final operation completes earlier.

3 Why Ant Colony Optimization?

JSSP becomes difficult quickly because every machine sequence interacts with every job route. Exact methods can be valuable, but this repository focuses on a metaheuristic because ACO provides a clear educational bridge between random construction, learning, and local improvement.

ACO is inspired by the way ants use pheromones to bias future path choices. In optimization, artificial ants construct solutions. Edges or choices that appear in better solutions receive more pheromone. Over time, the search becomes biased toward solution components that have historically produced good results.

In this project, an ant constructs a *job-order sequence*. If there are n jobs and m machines, the sequence has length $n \times m$. Each time job j appears in the sequence, the decoder schedules the next unscheduled operation of job j at the earliest feasible time. This representation is compact, easy to validate, and convenient for local search.

4 Current Repository Organization

The active code is organized around a clean separation between solver, tests, UI, legacy artifacts, and paper documentation.

Path	Purpose
<code>src/aco_jssp.py</code>	Modern Python 3 solver, generated instance creation, CLI, schedule decoder, local search, and JSON output.
<code>tests/test_aco_jssp.py</code>	Unit tests for precedence, machine non-overlap, deterministic seeds, generated instances, and complete schedules.
<code>docs/index.html</code>	Static browser UI and Gantt visualizer deployed through GitHub Pages.
<code>paper/</code>	Updated LaTeX report source and compiled PDF.
<code>legacy/</code>	Original 2017 prototype scripts retained for historical reference.
<code>ERRORS_AND_FIXES.md</code>	Review notes describing issues found in the historical implementation and fixes added in the modernized project.

This organization makes the repository easier to read: the active implementation is in `src/`, the UI is in `docs/`, the report is in `paper/`, and the old scripts are explicitly labeled as legacy.

5 Solver Design

5.1 Generated problem instances

The solver can generate reproducible random JSSP instances. The user chooses the number of jobs, number of machines, random seed, and duration range. Each generated job visits each machine exactly once, in a random route order. This keeps the model simple while letting the UI demonstrate larger and harder examples.

5.2 Job-order decoding

The core decoder turns a job-order sequence into a feasible active schedule. It tracks:

- the next operation to schedule for each job;
- the earliest time each job is ready;
- the earliest time each machine is available;
- the start and end time assigned to every operation.

When job j is selected, the decoder schedules the next operation of job j at:

$$s = \max(\text{jobReady}_j, \text{machineReady}_{\mu_{j,k}}).$$

This guarantees job precedence and machine capacity by construction.

5.3 Transition rule

At each construction step, an ant chooses among jobs that still have unscheduled operations. A job's attractiveness depends on pheromone and a heuristic score:

$$P(j) = \frac{\tau_{i,j}^\alpha \eta_j^\beta}{\sum_{\ell \in F} \tau_{i,\ell}^\alpha \eta_\ell^\beta},$$

where F is the feasible set, i is the previously selected job, $\tau_{i,j}$ is the pheromone value for moving from job i to job j , and η_j is the heuristic desirability.

The updated heuristic is remaining-work-aware:

$$\eta_j = \frac{1}{\max(\text{earliestFinish}_j + 0.15 \cdot \text{remainingWork}_j, 1)}.$$

This biases construction toward moves that look good immediately while also accounting for downstream work still left in the job.

6 Additional Techniques Implemented

The updated repository adds several techniques beyond the original prototype.

6.1 Separation of problem and algorithm parameters

The UI now clearly separates *problem parameters* from *algorithm parameters*. Problem parameters define the generated instance: jobs, machines, duration range, and instance seed. Algorithm parameters define how the solver searches: ants, iterations, local search budget, and algorithm seed. This separation makes the UI easier to understand and aligns with how optimization systems should be configured.

6.2 Exploration and exploitation

The solver mixes random weighted choice with a small amount of greedy exploitation. With probability q_0 , an ant chooses the feasible move with the highest weighted score. Otherwise, it samples probabilistically from the weighted distribution. This keeps the algorithm from being purely random while still preserving diversity.

6.3 Pheromone evaporation and reinforcement

After each iteration, pheromone is evaporated:

$$\tau_{i,j} \leftarrow (1 - \rho)\tau_{i,j}.$$

A numerical floor prevents pheromone values from disappearing completely. The solver then reinforces the iteration-best order and also applies an elite deposit to the global-best order:

$$\tau_{i,j} \leftarrow \tau_{i,j} + \frac{Q}{C_{\max}}$$

for transitions that appeared in a good sequence.

6.4 Pair-swap local search

After an ant constructs a full job-order sequence, the solver applies a small local search. It randomly swaps two positions in the sequence, decodes the candidate, and keeps the swap if makespan does not worsen. This is a simple neighborhood, but it improves the demonstration because constructed solutions can be refined before they influence pheromone.

6.5 Seeded reproducibility

The UI now separates instance seed from algorithm seed. Changing the instance seed changes the generated problem. Changing the algorithm seed changes the stochastic search path. This makes experimentation clearer and helps with testing.

7 Browser Visualizer

The browser visualizer is a static HTML page under `docs/index.html`. It runs directly on GitHub Pages without a backend. The UI exposes controls for both problem generation and algorithm behavior.

Control group	Controls
Problem parameters	Jobs, machines, minimum duration, maximum duration, instance seed.
Algorithm parameters	Ants, iterations, local search budget, algorithm seed.
Output	Makespan, number of operations, Gantt chart, job legend, schedule JSON.

The visualization is deliberately not overbuilt. It is meant to show the relationship between instance size, search effort, and resulting schedule structure. The Gantt chart makes machine conflicts and job sequencing visible immediately.

8 Testing and Validation

The test suite checks correctness properties that should hold regardless of stochastic performance:

- every generated schedule contains the required number of operations;

- each job’s operations respect route order;
- no two operations overlap on the same machine;
- generated instances are reproducible for a fixed seed;
- the solver is deterministic for a fixed algorithm seed;
- the solver can handle larger generated instances as a smoke test.

The tests can be run with:

```
python -m unittest discover -s tests -v
```

These tests validate feasibility and implementation stability. They do not prove global optimality.

9 Limitations

This project is an educational and demonstrative implementation. It does not yet include standard benchmark comparisons, optimality gaps, parallel ants, advanced critical-path neighborhoods, disjunctive graph visualizations, or exact MILP baselines. The browser UI is intentionally bounded so that it remains responsive.

The current ACO implementation also learns pheromone on transitions between job selections rather than richer operation-level or machine-level sequencing decisions. This keeps the method understandable, but richer pheromone models may improve solution quality.

10 Future Work

Useful extensions include:

1. Add benchmark instances such as FT06, LA, ORB, and ABZ families.
2. Add a MILP or CP-SAT baseline for small cases.
3. Add critical-path-based local search neighborhoods.
4. Add machine-specific pheromone matrices.
5. Add dispatching-rule hybridization, such as SPT, LPT, slack, or bottleneck-aware priority rules.
6. Export schedules to CSV and JSON from the UI.
7. Add runtime and convergence charts.
8. Add parallel ant construction for larger instances.

11 Conclusion

The updated ACO-JSSP repository is now more than a historical script. It is a small but coherent optimization demonstration: a Python solver, a generated instance model, a static browser UI, a Gantt visualizer, unit tests, GitHub Pages deployment, and a LaTeX-style technical report. The core lesson is that metaheuristics become much more useful when paired with clean representations, validation, reproducibility, and visualization.

References

- [1] M. Dorigo, V. Maniezzo, and A. Colorni. “Ant System: Optimization by a Colony of Cooperating Agents.” *IEEE Transactions on Systems, Man, and Cybernetics*, 26(1), 1996.
- [2] M. Dorigo and T. Stuetzle. *Ant Colony Optimization*. MIT Press, 2004.
- [3] M. R. Garey, D. S. Johnson, and R. Sethi. “The Complexity of Flowshop and Jobshop Scheduling.” *Mathematics of Operations Research*, 1(2), 1976.
- [4] J. Blazewicz, W. Domschke, and E. Pesch. “The Job Shop Scheduling Problem: Conventional and New Solution Techniques.” *European Journal of Operational Research*, 93(1), 1996.